2026.10

AIGC 智能创作工作流 Agent

从自然语言需求到可校验分镜与提示词

Python 3.13 LangGraph DeepSeek Pydantic SQLite Streamlit

> 不是聊天壳，而是有状态、可校验、可追溯的创作工作流。

答辩人：_____ | 代码仓库：gitee.com/qinboqun/aigc-agent

SYSTEM SNAPSHOT

20/20
评测行为符合预期

127
默认测试通过

4
确定性校验

2
最大修正次数

01 · 问题与目标

问题定义

为什么不能只做一个聊天框

聊天式生成的三个问题

Gap

1. 关键信息被模型静默补全，用户不知道哪些是确认过的。
2. 时长、场景、主体一致性无法验证，只能"看起来不错"。
3. 失败过程不可追溯，看不到调用了什么、错在哪一步。

项目目标

Goal

- 结构化需求 + Pydantic 数据契约
- 真实 Python 工具调用，而不是模型自评
- 四类确定性校验
- 最多两次修正，失败就停止
- SQLite 持久化、执行轨迹、可复盘

> 模型负责创作判断，Python 负责可规则化的验证。

3

核心问题

4

02 · 系统架构

分层，而不是把所有逻辑塞进页面

Architecture

Streamlit / CLI → CreativeProjectService → LangGraph → DeepSeek / FakeLLM + Python 工具 / 四类校验 → SQLite / 导出

交互层

UI

七个 Streamlit 页面；一句话进入主动引导；提示词包为主结果页；后台线程 + 队列。

Agent 层

State

状态、节点、条件路由；失败分支和修正分支显式表达。

工具层

Tools

分镜、提示词、时长、场景、一致性、提示词质量、修正、导出。

契约层

Pydantic

需求、镜头、分镜、校验报告；模型输出不能悄悄越界。

数据层

SQLite

项目、执行、轨迹、校验报告、修正历史；增量建表。

03 · AGENT 工作流

八个节点，两种失败，一条修正环路

LangGraph

analyze_requirements → plan_workflow → generate_storyboard → generate_prompts → validate_output → write_director_prompt

- **通过** → write_director_prompt → finalize_result（completed）
- **演示模式** → 跳过导演提示词，保留规则拼装兜底
- **失败且 revision_count < 2** → revise_output → 重新生成提示词 → 重新校验
- **达到上限或修正失败** → finalize_result（validation_failed / failed）

路由原则

Routing

- 需求/分镜失败：直接归档，不调用下游工具
- 校验失败：正常分支，不是异常
- 修正后：必须重新生成提示词并重新校验
- 导演提示词失败：明确 failed，不用兜底伪装成功

> 失败是工作流的一部分；系统不会因为"已经修正过"就跳过验证。

8

工作流节点

4

确定性校验

04 · 分镜与提示词

结构化分镜 + 导演提示词

Cinematography

每个镜头包含

Shot

- 时间、场景、叙事职责
- 起始动作、结束动作、主体状态
- 连续性锁、参考图用途
- 景别、机位、运镜、镜头、构图
- 灯光、色彩分级、转场、特效、屏幕方向
- 声音、负向提示词

提示词两层

Director

```
真实模式（DeepSeek）
整片导演提示词
+ 连续性圣经
+ 逐镜头提示词
+ 全片负向约束

演示模式（Python 兜底）
结构化字段 → 规则拼装
```

导演提示词由模型重写，不是字段标签拼接；Python 只做兜底。

9

景别枚举

05 · 四类校验

不询问模型“你觉得好不好”

Deterministic

时长

Time

负时长、重叠、空档、前导空档、总时长容差、非有限数字。

场景覆盖

Scene

缺失必需场景失败；重复场景警告；覆盖率统计。

主体一致性

Continuity

连续性锁、主体原样描述、车头/屏幕方向、文本方向矛盾。

提示词质量

Prompt

必要字段、矛盾约束、单镜头多运镜、模型负向提示词/语言差异。

错误 = 阻止通过；警告 = 需要确认，但不直接判定失败。

文本检查不能证明最终画面一致；语义边界写入报告。

06 · 修正循环

最多两次，然后明确停止

Max 2

1

Model

有模型

需求 + 当前分镜 + 方向审计 + 具体错误交给 DeepSeek，只修列出的问题。

2

Python

无模型

只有"时长是唯一失败项"时，才允许确定性缩放时间线。

3

Recheck

重新校验

修正后重新生成提示词，再跑四类校验；不能只改文本就宣布成功。

达到上限

validation_failed

保留最后一次校验报告，错误信息包含"已达到自动修正上限 2 次"。

修正工具失败

failed

模型返回非法 JSON、网络失败或结果无法解析；系统停止，不伪造成功。

07 · 数据与可追溯

SQLite

每一步都留下真实记录

七张表

Schema

表	内容
projects	需求、任务计划、分镜
executions	执行状态、时间、错误
execution_traces	节点、工具、参数、摘要
validation_reports	四类校验聚合报告
revision_history	修正原因、前后摘要、用量
creative_sessions	对话、字段来源、方案状态
creative_assets	可复用车辆资产

轨迹字段

Trace

- 节点名、工具名、工具参数
- 输入摘要、输出摘要
- 状态、错误、时间
- 修正编号、原因、前后摘要
- 模型调用次数、token 用量

轨迹来自真实函数调用和 LangGraph 流式状态。

08 · STREAMLIT 工作台

主动引导 + 提示词优先，七个页面

Product UI

1. **创作工作台**
   一句话输入、动态追问、专业推荐、方案确认、增量修改。

2. **提示词包**
   整片复制、逐镜头图片/视频/负向提示词、TXT 下载。

3. **分镜详情**
   分镜表、摄影参数、连续性约束。

4. **校验报告**
   四类状态；通过/警告/失败/未执行。

5. **执行过程**
   真实流式事件、工具轨迹、修正记录。

6. **项目历史**
   SQLite 读取、搜索、筛选、重新载入。

7. **导出**
   Markdown/JSON、唯一文件名、预览。

09 · 20 条评测

Evaluation

20 / 20 是行为符合预期，不是全部成功

20/20

用例期望匹配

8 条确定性工作流 · 9 条脚本化 LLM 工作流 · 3 条工具级；另有 127 项默认测试

13

completed

1

validation_failed

3

failed

3

工具级

10 · 工程与安全

Engineering

能跑、能测，也要能守住边界

工程

Build

- Pydantic 契约；模型输出不能越界
- Pytest：单元、集成、Streamlit AppTest
- 20 条评测 + 自动 Markdown/JSON 报告
- SQLite 增量建表；历史只读
- 导出唯一文件名，不覆盖

安全

Secure

- API Key 只从环境变量读取
- .env 与 .history/ 不进入 Git
- .env.example 不含真实密钥
- 错误信息脱敏；日志/轨迹不含密钥
- 评测报告扫描密钥样式字符串

> 未执行的命令不说执行，未通过的测试不说通过，未配置的 API 不说可用。

127

默认测试通过

Scope

把边界说清楚，才有下一步

当前边界

Now

- 不直接生成图片/视频，只生成分镜和提示词
- 确定性检查不能保证最终画面一致
- 英文模型提示词方言转换未实现
- 图片不做视觉识别；已有完整分镜不自动解析
- 评测默认用 FakeLLM，不代表真实模型质量
- 无登录、多租户、远程部署

下一步

Next

- 英文提示词适配层
- 真实 API 分层评测
- 人工审美评分与失败案例库
- 持久化任务队列，避免状态悬挂
- 接入实际生成 API，保留生成后验收

让每一步可检查、可记录、可复现；无法解决时，明确失败。

谢谢各位老师，欢迎提问。